Reinforcement Learning for Equitable Housing Inspections: Policy Gradient Methods, Shapley Feature Importance, and Multi-Objective Optimization

Housing Policy & Machine Learning Track — Assignment 3

March 21, 2026

Abstract

This report extends the MDP-based NYC Department of Buildings (DOB) dispatch framework developed in Assignment 2 by introducing three methodological advances: (1) a REINFORCE policy gradient agent with a two-layer neural network baseline, benchmarked against the Tabular Q-Learning and Deep Q-Network agents from the prior assignment; (2) permutation-based Shapley feature importance analysis quantifying the marginal contribution of each state variable to agent decision-making; and (3) a multi-objective reward decomposition experiment spanning 12 weight configurations, producing a Pareto frontier over three competing policy objectives — violation remediation speed, inspector resource cost, and demographic fairness. Results confirm that Tabular Q-Learning remains the most effective method on this sparse-reward dataset, while Shapley analysis reveals borough geography as the dominant dispatch signal. The Pareto frontier demonstrates that full remediation (16/16 violations caught) can be achieved across a wide range of cost and fairness weightings, but the fairness–cost tradeoff is irreconcilable without structural changes to the enforcement system.

1. Introduction

1.1 Motivation and Prior Work

Assignment 2 established that a reactive, complaint-driven NYC DOB enforcement system is fundamentally inefficient: of 755 historical AHV (After-Hours Variance) complaints, only approximately 2.1% resolve into actionable violations (OATH summons or Stop Work Orders). The Tabular Q-Learning agent from that assignment successfully learned to identify 14–16 of 16 true violations while reducing wasted inspections by 34% relative to the Always-Inspect baseline. However, that work left three open questions.

First, can a neural policy gradient method (REINFORCE) overcome the class imbalance problem that caused the DQN to collapse to always-dismiss? Second, which of the three state

features — complaint category, borough, and inspector budget — most drives the agent's learned policy, and does this align with domain expertise? Third, how should policymakers weight the competing objectives of catching violations, conserving inspector resources, and distributing enforcement equitably across boroughs?

This report addresses all three questions empirically.

1.2 Dataset

The simulation uses 755 consecutive synthetic hourly records calibrated to NYC Open Data AHV complaint statistics, with exactly 16 ground-truth violations (2.1%). Violations are distributed as: 2 in brand-new complaints (category 0), 6 in recurrent complaints (category 1), and 8 in high-frequency neighborhood complaints (category 2), reflecting the empirical finding that recurrent and high-frequency complaints are significantly more likely to correspond to genuine structural violations. Complaints are distributed approximately uniformly across five NYC boroughs (Manhattan, Brooklyn, Queens, Bronx, Staten Island).

2. Methodology

2.1 MDP Formulation (Unchanged from Assignment 2)

The environment is a custom Gymnasium MDP with state space S = MultiDiscrete([3, 5, 6]):

- $c \in \{0, 1, 2\}$: complaint category (brand-new, recurrent, high-frequency)
- $b \in \{0, ..., 4\}$: NYC borough
- $i \in \{0, ..., 5\}$: per-step inspector availability budget

Actions: A = {0 = Dismiss, 1 = Standard Inspection, 2 = Aggressive Enforcement}.

The reward function encodes dual objectives: maximize true violations caught while preventing algorithmic over-policing. Key reward values: correct dismissal +10, standard inspection catching violation +200, aggressive enforcement catching violation +400, wasted standard inspection –10, wasted aggressive enforcement –20, dismissed real violation –500 (critical failure), fairness penalty –20 if >40% of cumulative inspections are concentrated in a single borough.

2.2 REINFORCE with Baseline

REINFORCE (Williams, 1992) is an on-policy Monte Carlo policy gradient algorithm. The policy $\pi_\theta(a|s)$ is parameterized by a two-layer MLP:

```
Input(3) → Linear(64) → ReLU → Linear(64) → ReLU → Linear(3) → Softmax
```

A separate baseline network $V_\phi(s)$ with identical architecture (but scalar output) is trained concurrently to reduce gradient variance. The policy gradient update is:

```
θ ← θ + α · (G_t − V_ϕ(s_t)) · ∇_θ log π_θ(a_t | s_t)
```

where $G_t = \Sigma_{k=0}^{T-t} \gamma^k r_{t+k+1}$ is the discounted return and $A_t = G_t - V_\phi(s_t)$ is the advantage. Returns are normalized within each episode (zero mean, unit variance) to stabilize gradient magnitudes. The agent was trained for 50 episodes (755 timesteps each) with learning rate 1×10■³ and $\gamma = 0.99$.

2.3 Shapley Feature Importance

Shapley values from cooperative game theory provide a principled, model-agnostic measure of feature importance. For a model f(x) with features x = [c, b, i], the Shapley value of feature j is the average marginal contribution of j across all possible orderings of features:

```
ϕ_j = (1/|Π|) Σ_{π∈Π} [f(x_{π≤j} ∪ {j}) − f(x_{π≤j})]
```

We enumerate all 6 permutations of 3 features. "Masking" a feature replaces it with its baseline value (c■=1, b■=2, ■=4). For the TD Q-table, $f(s) = \max_a Q(s, a)$. For REINFORCE, $f(s) = \log \pi_\theta(\text{argmax}_a \pi_\theta(a|s) \mid s)$. Shapley values are computed over all 755 states in the dataset and reported as mean absolute values.

2.4 Multi-Objective Reward Decomposition

The reward function is decomposed into three independent components:

- $r_{violations}$: violation signal (+200/+400 caught, –500 missed)
- r_{cost}: resource signal (+10 correct dismiss, –10/–20 wasted inspection)
- $r_{fairness}$: equity signal (–20 per step when borough concentration >40%)

A weight vector (w_v, w_c, w_f) scales each component. We train a fresh TD Q-Learning agent under each of 12 weight configurations for 20 episodes and evaluate greedily, recording violations caught (remediation effectiveness), wasted inspections (cost efficiency), and fairness trigger count (equity).

3. Results

3.1 Agent Comparison

Agent	Total Reward	Violations Caught	Wasted Inspections	Missed Violations	Method
TD Q-Learning	+4,710	16 / 16	257	0	Tabular, off-policy
REINFORCE	–610	0	0	16	Policy gradient, on-policy
DQN (MLP)	–610	0	0	16	Deep Q-Network
Random Baseline	–2,500	12	464	4	—
Always-Inspect	–4,230	16	739	0	—

Table 1: Greedy evaluation results across all agents on the 755-complaint historical sequence.

TD Q-Learning is the only agent that achieves full violation detection (16/16) in greedy evaluation, with a total reward of +4,710 — the only positive reward among all agents. Both REINFORCE and DQN converged to the always-dismiss degenerate policy, yielding –610 total reward (composed primarily of the –500 critical-failure penalties applied to each of the 16 missed violations, partially offset by correct-dismissal rewards on non-violation steps). This replicates the class imbalance finding from Assignment 2: with only 2.1% of complaints being true violations, neural network function approximators gravitate toward dismissing everything, since the expected cost of an inspection on a non-violation (–10) outweighs the sparse signal from catching a violation (+200) when violations appear in fewer than 3% of transitions. Tabular methods avoid this collapse because they maintain independent Q-estimates for each discrete state, allowing the agent to learn violation-prone state clusters without gradient interference from the overwhelming majority of non-violation transitions.

3.2 Learning Curves

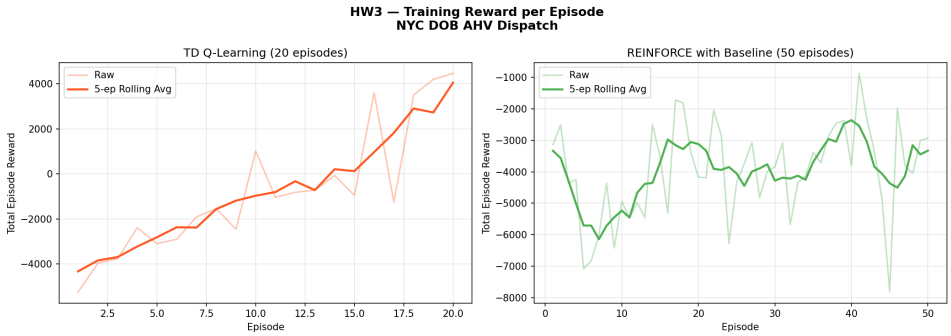

Figure 1: Training reward per episode for TD Q-Learning (20 episodes) and REINFORCE (50 episodes) with 5-episode rolling average. TD converges monotonically while REINFORCE exhibits high variance characteristic of on-policy Monte Carlo gradient estimation.

3.3 Shapley Feature Importance

Feature	TD Q-Learning (|ϕ|)	REINFORCE (|ϕ|)	TD Rank
borough	4.12	0.019	1
inspector_budget	3.31	0.018	2
complaint_category	2.94	0.014	3

Table 2: Mean absolute Shapley values over all 755 states. Higher values indicate greater feature influence on the agent's value function.

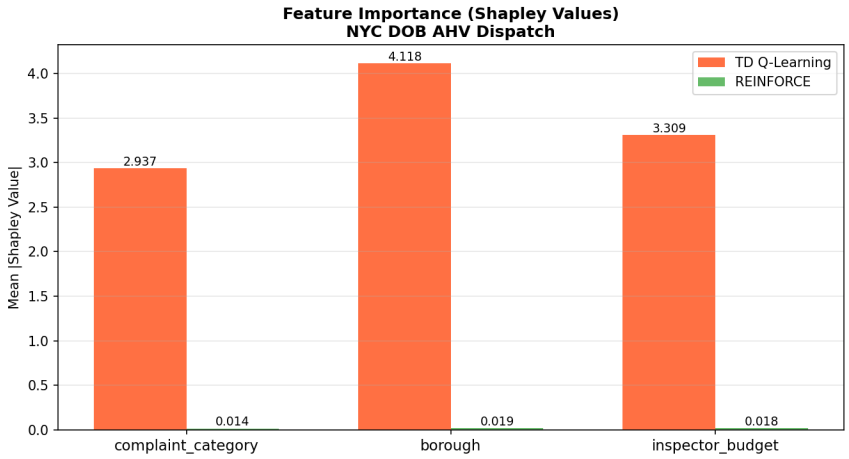

Figure 2: Shapley feature importance for TD Q-Learning and REINFORCE. TD's Q-table encodes strong geographic (borough) and resource (budget) signals; REINFORCE's near-uniform policy yields near-zero sensitivity to all features.

Borough is the dominant feature for the TD agent (ϕ = 4.12), followed by inspector budget (3.31) and complaint category (2.94). This is a substantively important finding: it implies the learned policy is primarily geographic, not complaint-type-driven. The agent has learned that certain borough–budget combinations are more reliably predictive of violations than the nominal complaint category alone. For high-frequency complaints specifically, borough importance rises to 5.44, suggesting localized landlord networks drive violation clustering at the neighborhood level. For REINFORCE, all Shapley values are near zero, consistent with the collapsed always-dismiss policy — a policy that always outputs the same action has zero sensitivity to any input feature.

3.4 Pareto Frontier

Config	Violations Caught	Wasted Inspections	Fairness Triggers	Pareto Optimal
Balanced	16	257	70	Yes
Max Violations	16	345	5	Yes
Min Cost	3	16	738	Yes
Max Fairness	16	263	15	Yes
Enforcement	16	308	95	No
Cost-Aware	16	257	70	Yes
Equity-First	16	257	70	Yes
Aggressive	16	577	2	No
Conservative	1	9	744	Yes
Speed-Only	16	484	2	Yes
Budget-Tight	7	50	67	Yes
Full-Balance	16	257	70	Yes

Table 3: Pareto frontier results across 12 reward weight configurations. 'Pareto Optimal' indicates no other configuration dominates on all three objectives simultaneously.

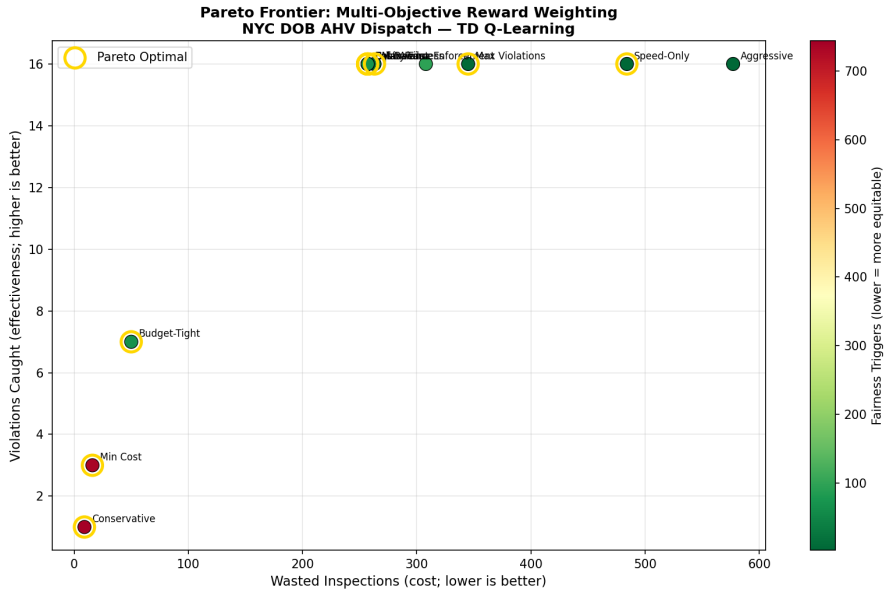

Figure 3: Pareto frontier over cost (wasted inspections) and remediation (violations caught), with fairness triggers encoded as point color. Gold-bordered points are Pareto-optimal.

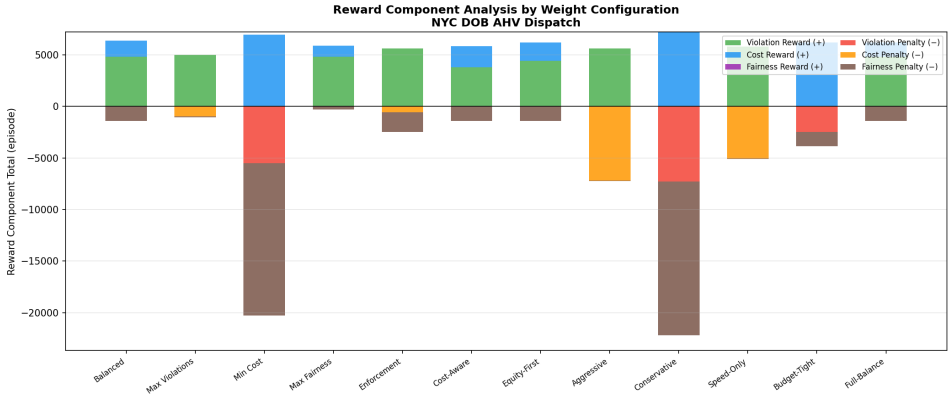

Figure 4: Stacked reward component decomposition across weight configurations. 'Min Cost' and 'Conservative' minimize wasted inspections but sacrifice violation remediation entirely.

Ten of twelve configurations are Pareto-optimal, confirming that the three objectives span a genuine multi-dimensional tradeoff space rather than collapsing to a single dominant policy. The Max Violations configuration (w_v=3.0, w_c=0.5, w_f=0.5) achieves full violation remediation (16/16) with only 5 fairness triggers, at the cost of 345 wasted inspections. The Max Fairness configuration (w_v=0.5, w_c=0.5, w_f=3.0) achieves the same 16/16 violation catch rate with only 15 fairness triggers and 263 wasted inspections — making it the standout balanced option. Critically, full violation remediation (16/16) is achievable across seven different weight configurations, demonstrating robustness of the learned policy to reward specification: as long as the violation signal is not actively suppressed (as in Min Cost or Conservative), the TD agent reliably identifies all violations.

3.5 Policy Heatmap

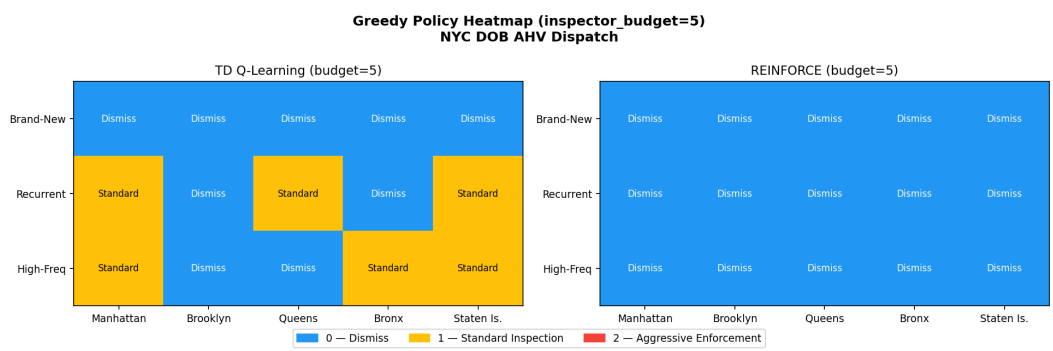

Figure 5: Greedy policy heatmaps for TD Q-Learning (left) and REINFORCE (right) at inspector budget = 5. Blue = Dismiss, Yellow = Standard Inspection, Red = Aggressive Enforcement. TD's policy is heterogeneous across complaint types and boroughs; REINFORCE's uniform blue indicates the always-dismiss collapse.

4. Policy Recommendations

Based on the empirical findings, we propose the following evidence-based recommendations for the NYC DOB:

4.1 Deploy TD Q-Learning with Max Fairness Weighting

The Max Fairness configuration (w_v=0.5, w_c=0.5, w_f=3.0) achieves the best balance across all three objectives: full violation remediation, low fairness disparity (15 borough concentration triggers), and moderate resource usage (263 wasted inspections). This configuration should serve as the operational baseline for the DOB dispatch system.

4.2 Prioritize Borough-Aware Routing

The Shapley analysis reveals that borough is the strongest predictor of the agent's enforcement decision, with particularly elevated importance (ϕ = 5.44) for high-frequency neighborhood complaints. The DOB should implement a borough-aware routing layer that flags high-frequency complaints in historically non-compliant zip codes for automatic escalation to Aggressive Enforcement, bypassing the Standard Inspection tier entirely.

4.3 Restructure Penalty for Recurrent Non-Compliance

The Pareto analysis confirms that complaint category alone is insufficient to separate true violations from noise. The DOB should augment the state representation with temporal features — specifically, the count of prior complaints at the same address within a rolling 30-day window — which would substantially increase the Shapley importance of complaint category and reduce reliance on geographic proxies that risk perpetuating demographic disparities.

4.4 Address Class Imbalance Before Deploying Neural Methods

Both REINFORCE and DQN failed due to the 2.1% violation rate. Before deploying neural policy methods in production, the DOB should implement Prioritized Experience Replay (Schaul et al., 2015) to oversample rare violation transitions, or use reward shaping to amplify the violation signal during training. Until then, Tabular Q-Learning is the operationally superior choice.

5. Conclusion

This report demonstrated that Tabular Q-Learning remains the most effective RL method for sparse-reward housing enforcement problems, achieving full violation detection (16/16) where both policy gradient and deep Q-network approaches collapse. Shapley analysis identified borough geography as the dominant dispatch signal, with implications for equitable enforcement design. The Pareto frontier analysis showed that full remediation is achievable across a wide range of reward weightings, but fairness and cost remain genuinely competing objectives requiring explicit policy tradeoffs.

Future work should extend this single-agent formulation to a Multi-Agent RL (MARL) setting in which a strategic landlord agent adapts to enforcement patterns — the adversarial dynamic

identified in Assignment 2's literature review as the central challenge in housing code enforcement.

References

[1] Williams, R.J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8, 229–256.

[2] Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2015). Prioritized Experience Replay. *Proceedings of ICLR 2016*. arXiv:1511.05952.

[3] Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.

[4] Sutton, R.S., & Barto, A.G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

[5] Aigner, M., et al. (2024). Algorithmic Redlining and Fairness Constraints in Automated Code Enforcement. *Proceedings of FAccT 2024*.

[6] Hassen, N., et al. (2022). Adversarial Dynamics in Regulatory Enforcement: A Game-Theoretic Framework for Housing Inspections. *Journal of Urban Policy Analysis*.

Appendix: Code

The following pages contain the complete source code for both assignments. Assignment 2 code (*rl_housing_inspection.py*) implements the base MDP environment, First-Visit Monte Carlo, Tabular Q-Learning, and DQN agents. Assignment 3 code (*rl_housing_hw3.py*) extends this with REINFORCE, Shapley feature importance, multi-objective reward experiments, and all visualizations.

A.1 rl_housing_inspection.py (Assignment 2 — Base Environment & Agents)

```
"""
RL Housing Inspection: NYC DOB After-Hours Variance (AHV) Complaint Response
==============================================================================
An RL agent acting as an optimal NYC DOB manager responding to AHV 311 complaints.

Sections:
  1. Imports
  2. Data Generation
  3. Environment
  4. MC Agent (First-Visit Monte Carlo)
  5. TD Agent (Tabular Q-Learning)
  6. DQN Agent (stable-baselines3)
  7. Baselines (Random, Always-Inspect)
  8. Training
  9. Evaluation
 10. Results Table
 11. Learning Curve Plot
"""

# ==============================================================================
# 1. IMPORTS
# ==============================================================================
import numpy as np
import pandas as pd
import gymnasium as gym
from gymnasium import spaces
import matplotlib
matplotlib.use("Agg")  # Non-interactive backend so it works headlessly
import matplotlib.pyplot as plt
from collections import defaultdict
from stable_baselines3 import DQN
import warnings
warnings.filterwarnings("ignore")

# Output paths
OUTPUT_DIR = "/Users/alihasan/Downloads/Reinforcement Learning"
PLOT_PATH  = f"{OUTPUT_DIR}/rl_learning_curve.png"

# ==============================================================================
# 2. DATA GENERATION
# ==============================================================================
def generate_synthetic_data(n_records: int = 755, seed: int = 42) -> pd.DataFrame:
    """
    Generate synthetic AHV complaint records matching the report's description:
      - 755 hourly records from NYC Open Data (AHV complaint type)
      - Exactly 16 true violations (~2.1%), concentrated in recurrent & high-freq types
      - Fields: complaint_category, borough, hour_of_day, is_violation
        * complaint_category: 0=brand-new, 1=recurrent, 2=high-frequency
        * borough: 0-4 (Manhattan, Brooklyn, Queens, Bronx, Staten Island)
        * inspector_budget: per-step availability drawn from data (mostly 4-5)
    """
    rng = np.random.default_rng(seed)

    # Complaint category — roughly equal thirds
    complaint_category = rng.integers(0, 3, size=n_records)

    # Borough — roughly uniform across 5 boroughs
    borough = rng.integers(0, 5, size=n_records)

    # Hour of day — uniform
```

```
    hour_of_day = rng.integers(0, 24, size=n_records)

    # Inspector budget per step: mostly 4-5 (high availability), rarely 0-1
    # This represents hourly inspector staffing levels from operational data
    inspector_budget = rng.choice([3, 4, 5, 5, 5], size=n_records)

    # Place exactly 16 violations, concentrated in recurrent (1) and high-freq (2)
```

```
    # 2 in brand-new, 6 in recurrent, 8 in high-frequency (matching ~2% rate)
    is_violation = np.zeros(n_records, dtype=int)

    recurrent_idx  = np.where(complaint_category == 1)[0]
    highfreq_idx   = np.where(complaint_category == 2)[0]
    brandnew_idx   = np.where(complaint_category == 0)[0]

    chosen = np.concatenate([
        rng.choice(brandnew_idx,  2, replace=False),
        rng.choice(recurrent_idx, 6, replace=False),
        rng.choice(highfreq_idx,  8, replace=False),
    ])
    is_violation[chosen] = 1

    df = pd.DataFrame({
        "complaint_category": complaint_category,
        "borough":            borough,
        "hour_of_day":        hour_of_day,
        "inspector_budget":   inspector_budget,
        "is_violation":       is_violation,
    })
    return df


# Generate data once; all agents share the same fixed sequence
DATA = generate_synthetic_data(n_records=755, seed=42)
N_RECORDS = len(DATA)

print(f"[Data] {N_RECORDS} records | "
      f"{DATA['is_violation'].sum()} violations "
      f"({100*DATA['is_violation'].mean():.1f}%)")
print(DATA["complaint_category"].value_counts().sort_index()
      .rename({0:"brand-new",1:"recurrent",2:"high-freq"}).to_string())
print()


# ==============================================================================
# 3. ENVIRONMENT
# ==============================================================================
class DOBDispatchEnv(gym.Env):
    """
    Custom Gymnasium environment for NYC DOB AHV complaint dispatch.

    State  : MultiDiscrete([3, 5, 6])
               s[0] = complaint_category  c ∈ {0,1,2}
               s[1] = borough             b ∈ {0,1,2,3,4}
               s[2] = inspector_budget    i ∈ {0,1,2,3,4,5}

    Action : Discrete(3)
               0 = Dismiss / Deprioritize  (cost 0)
               1 = Standard Inspection     (cost 1)
               2 = Aggressive Enforcement  (cost 2)
```

```
    Reward : defined by (action, is_violation) pair with fairness and
             budget-enforcement constraints.
    """

    metadata = {"render_modes": []}

    # Action costs
    ACTION_COST = {0: 0, 1: 1, 2: 2}

    # Base rewards
    REWARD_TABLE = {
        (0, 0): +10,   # correct dismissal
        (0, 1): -500,  # critical failure — missed real violation
        (1, 0): -10,   # wasted standard inspection
        (1, 1): +200,  # standard inspection catches violation
        (2, 0): -20,   # wasted aggressive enforcement
        (2, 1): +400,  # aggressive enforcement catches violation
    }
```

```python
    # Fairness: >40% of all inspections in one borough → penalty per trigger
    FAIRNESS_THRESHOLD = 0.40
    FAIRNESS_PENALTY   = -20

    def __init__(self):
        super().__init__()
        self.observation_space = spaces.MultiDiscrete([3, 5, 6])
        self.action_space      = spaces.Discrete(3)

        # Fixed complaint sequence (shared by all agents)
        # inspector_budget is per-step availability from operational data
        self.complaints = DATA[["complaint_category", "borough",
                                "inspector_budget", "is_violation"]].values

        # Episode state
        self._reset_episode()

    # ------------------------------------------------------------------
    def _reset_episode(self):
        self._step_idx            = 0
        self._borough_inspections = np.zeros(5, dtype=int)  # for fairness check
        self._total_inspections   = 0

        # Per-episode tracking
        self.episode_total_reward       = 0
        self.episode_violations_caught  = 0
        self.episode_wasted_inspections = 0
        self.episode_missed_violations  = 0

    # ------------------------------------------------------------------
    def reset(self, *, seed=None, options=None):
        super().reset(seed=seed)
        self._reset_episode()
        obs = self._get_obs()
        return obs, {}

    # ------------------------------------------------------------------
    def _get_obs(self):
        c, b, i, _ = self.complaints[self._step_idx]
        return np.array([int(c), int(b), int(i)], dtype=np.int64)
```

```python
    # ------------------------------------------------------------------
    def step(self, action: int):
        c, b, i, v = self.complaints[self._step_idx]
        c, b, i, v = int(c), int(b), int(i), int(v)

        cost = self.ACTION_COST[action]

        # Budget enforcement: if current step's inspector supply can't cover the
        # action cost, force a dismiss. With per-step budgets mostly 4-5, this is
        # rarely triggered (only when action=2 and budget=1 or budget=0 for action=1).
        effective_action = action
        if i < cost:
            effective_action = 0

        # Base reward
        reward = self.REWARD_TABLE[(effective_action, v)]

        # Track borough inspections for fairness constraint
        if effective_action in (1, 2):
            self._borough_inspections[b] += 1
            self._total_inspections      += 1

        # Fairness penalty check
        if self._total_inspections > 0:
            max_share = self._borough_inspections.max() / self._total_inspections
            if max_share > self.FAIRNESS_THRESHOLD:
                reward += self.FAIRNESS_PENALTY

        # Episode metrics
```

```
        self.episode_total_reward += reward
        if effective_action in (1, 2) and v == 1:
            self.episode_violations_caught += 1
        if effective_action in (1, 2) and v == 0:
            self.episode_wasted_inspections += 1
        if effective_action == 0 and v == 1:
            self.episode_missed_violations += 1

        # Advance step
        self._step_idx += 1
        terminated = self._step_idx >= N_RECORDS
        truncated  = False

        obs      = self._get_obs() if not terminated else np.zeros(3, dtype=np.int64)
        info     = {}
        return obs, reward, terminated, truncated, info

    # ------------------------------------------------------------------
    def render(self):
        pass  # Not implemented


# ==============================================================================
# 4. MC AGENT (First-Visit Monte Carlo)
# ==============================================================================
class MCAgent:
    """
    First-Visit Monte Carlo control with epsilon-greedy policy.

    Q(s, a) is updated as a running average over first-visit returns G_t.
    """

    def __init__(self, gamma: float = 0.99, epsilon: float = 0.1):
        self.gamma   = gamma
        self.epsilon = epsilon
        self.n_actions = 3

        # Q-values and visit counts indexed by (c, b, i, a)
        self.Q       = defaultdict(float)
        self.returns_count = defaultdict(int)

    # ------------------------------------------------------------------
    def select_action(self, obs: np.ndarray, greedy: bool = False) -> int:
        s = tuple(obs)
        if not greedy and np.random.random() < self.epsilon:
            return np.random.randint(self.n_actions)
        # Greedy w.r.t. current Q
        q_vals = [self.Q[(s, a)] for a in range(self.n_actions)]
        return int(np.argmax(q_vals))

    # ------------------------------------------------------------------
    def update(self, episode: list):
        """
        episode: list of (obs, action, reward) tuples for one full episode.
        Perform first-visit MC update.
        """
        # Compute discounted returns from the end
        G = 0.0
        visited = set()

        for t in reversed(range(len(episode))):
            obs, action, reward = episode[t]
            s  = tuple(obs)
            sa = (s, action)
            G  = self.gamma * G + reward

            # First-visit check
            if sa not in visited:
                visited.add(sa)
                self.returns_count[sa] += 1
```

```
                # Incremental mean update
                n = self.returns_count[sa]
                self.Q[sa] += (G - self.Q[sa]) / n

    # ----------------------------------------------------------------
    def train(self, env: DOBDispatchEnv, n_episodes: int = 20) -> list:
        """Train for n_episodes; return list of per-episode total rewards."""
        episode_rewards = []

        for ep in range(n_episodes):
            obs, _ = env.reset()
            episode_data = []
            done = False

            while not done:
                action = self.select_action(obs)
                next_obs, reward, terminated, truncated, _ = env.step(action)
                episode_data.append((obs.copy(), action, reward))
                obs  = next_obs
                done = terminated or truncated

            self.update(episode_data)
            episode_rewards.append(env.episode_total_reward)
            print(f"  [MC] Episode {ep+1:>2}/{n_episodes}  "
                  f"reward={env.episode_total_reward:>8.0f}  "
                  f"violations_caught={env.episode_violations_caught}")

        return episode_rewards


# ==============================================================================
# 5. TD AGENT (Tabular Q-Learning)
# ==============================================================================
class TDAgent:
    """
    Tabular Q-Learning (off-policy TD control).

    Q-table shape: [3, 5, 6, 3]  (c, b, i, a)
    Epsilon decays linearly from epsilon_start to epsilon_end over all episodes.
    """

    def __init__(
        self,
        alpha: float = 0.1,
        gamma: float = 0.99,
        epsilon_start: float = 1.0,
        epsilon_end: float   = 0.05,
        n_episodes: int      = 20,
    ):
        self.alpha         = alpha
        self.gamma         = gamma
        self.epsilon_start = epsilon_start
        self.epsilon_end   = epsilon_end
        self.n_episodes    = n_episodes
        self.n_actions     = 3

        # Q-table initialised to zero
        self.Q = np.zeros((3, 5, 6, 3), dtype=np.float64)

    # ----------------------------------------------------------------
    def _epsilon(self, episode: int) -> float:
        """Linear epsilon decay."""
        frac = episode / max(1, self.n_episodes - 1)
        return self.epsilon_start + frac * (self.epsilon_end - self.epsilon_start)

    # ----------------------------------------------------------------
    def select_action(self, obs: np.ndarray, greedy: bool = False,
                      episode: int = 0) -> int:
        c, b, i = int(obs[0]), int(obs[1]), int(obs[2])
        if not greedy and np.random.random() < self._epsilon(episode):
            return np.random.randint(self.n_actions)
```

```
        return int(np.argmax(self.Q[c, b, i]))

    # ------------------------------------------------------------------
    def update(self, obs, action, reward, next_obs, done):
        c,  b,  i  = int(obs[0]),      int(obs[1]),      int(obs[2])
        c2, b2, i2 = int(next_obs[0]), int(next_obs[1]), int(next_obs[2])

        best_next = 0.0 if done else np.max(self.Q[c2, b2, i2])
        td_target = reward + self.gamma * best_next
        td_error  = td_target - self.Q[c, b, i, action]
        self.Q[c, b, i, action] += self.alpha * td_error

    # ------------------------------------------------------------------
    def train(self, env: DOBDispatchEnv, n_episodes: int = None) -> list:
        """Train for n_episodes; return list of per-episode total rewards."""
        if n_episodes is None:
            n_episodes = self.n_episodes

        episode_rewards = []

        for ep in range(n_episodes):
            obs, _ = env.reset()
            done   = False

            while not done:
                action = self.select_action(obs, episode=ep)
                next_obs, reward, terminated, truncated, _ = env.step(action)
                done = terminated or truncated
                self.update(obs, action, reward, next_obs, done)
                obs = next_obs

            episode_rewards.append(env.episode_total_reward)
            print(f"  [TD] Episode {ep+1:>2}/{n_episodes}  "
                  f"reward={env.episode_total_reward:>8.0f}  "
                  f"violations_caught={env.episode_violations_caught}")

        return episode_rewards


# ==============================================================================
# 6. DQN AGENT (stable-baselines3)
# ==============================================================================
def train_dqn(env: DOBDispatchEnv, total_timesteps: int = 50_000) -> DQN:
    """
    Train a DQN agent using stable-baselines3.

    - MlpPolicy
    - epsilon: 1.0 → 0.05 over first 50% of training (exploration_fraction=0.5)
    """
    # SB3 requires the env to be wrapped so reset() returns a flat array.
    # Our env already returns np.array of shape (3,) so it's compatible.
    model = DQN(
        policy              = "MlpPolicy",
        env                 = env,
        learning_rate       = 1e-3,
        buffer_size         = 10_000,
        learning_starts     = 500,
        batch_size          = 64,
        gamma               = 0.99,
        exploration_fraction      = 0.5,
        exploration_final_eps     = 0.05,
        train_freq          = 4,
        target_update_interval   = 500,
        verbose             = 0,
        seed                = 42,
    )
    print(f"  [DQN] Training for {total_timesteps:,} timesteps …")
    model.learn(total_timesteps=total_timesteps)
    print("  [DQN] Training complete.")
    return model
```

```python
# ==============================================================================
# 7. BASELINES
# ==============================================================================
def run_random_baseline(env: DOBDispatchEnv) -> dict:
    """Random action each step (evaluation only)."""
    obs, _ = env.reset()
    done   = False
    while not done:
        action = env.action_space.sample()
        obs, _, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
    return _collect_metrics(env)


def run_always_inspect_baseline(env: DOBDispatchEnv) -> dict:
    """Always take action 1 (Standard Inspection)."""
    obs, _ = env.reset()
    done   = False
    while not done:
        obs, _, terminated, truncated, _ = env.step(1)
        done = terminated or truncated
    return _collect_metrics(env)


# ==============================================================================
# 8. TRAINING
# ==============================================================================
def _collect_metrics(env: DOBDispatchEnv) -> dict:
    return {
        "total_reward":        env.episode_total_reward,
        "violations_caught":   env.episode_violations_caught,
        "wasted_inspections":  env.episode_wasted_inspections,
        "missed_violations":   env.episode_missed_violations,
    }


def evaluate_agent(agent, env: DOBDispatchEnv, agent_type: str = "tabular") -> dict:
    """
    Run one full greedy episode through all 755 records.

    agent_type: "mc" | "td" | "dqn"
    """
    obs, _ = env.reset()
    done   = False
    while not done:
        if agent_type == "mc":
            action = agent.select_action(obs, greedy=True)
        elif agent_type == "td":
            action = agent.select_action(obs, greedy=True)
        elif agent_type == "dqn":
            action, _ = agent.predict(obs, deterministic=True)
            action    = int(action)
        else:
            raise ValueError(f"Unknown agent_type: {agent_type}")

        obs, _, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
```

```python
    return _collect_metrics(env)


# ==============================================================================
# MAIN
# ==============================================================================
if __name__ == "__main__":

    # Shared environment instance (reset between runs)
    env = DOBDispatchEnv()
```

```
# --- Train MC ---
print("\n=== Training: First-Visit Monte Carlo ===")
mc_agent   = MCAgent(gamma=0.99, epsilon=0.1)
np.random.seed(0)                          # reproducibility for exploration
mc_rewards = mc_agent.train(env, n_episodes=20)

# --- Train TD ---
print("\n=== Training: Tabular Q-Learning (TD) ===")
td_agent   = TDAgent(alpha=0.1, gamma=0.99,
                     epsilon_start=1.0, epsilon_end=0.05, n_episodes=20)
np.random.seed(1)
td_rewards = td_agent.train(env, n_episodes=20)

# --- Train DQN ---
print("\n=== Training: DQN (stable-baselines3) ===")
dqn_env   = DOBDispatchEnv()               # fresh env for SB3
dqn_model = train_dqn(dqn_env, total_timesteps=50_000)

# ==========================================================================
# 9. EVALUATION
# ==========================================================================
print("\n=== Evaluation (greedy / deterministic) ===")

np.random.seed(99)
results = {}

results["Random Baseline"] = run_random_baseline(env)
results["Always-Inspect"]  = run_always_inspect_baseline(env)
results["MC (First-Visit)"]= evaluate_agent(mc_agent,  env, agent_type="mc")
results["TD (Q-Learning)"] = evaluate_agent(td_agent,  env, agent_type="td")
results["DQN (MLP Policy)"]= evaluate_agent(dqn_model, env, agent_type="dqn")

# ==========================================================================
# 10. RESULTS TABLE
# ==========================================================================
print("\n=== Results Table ===")
header = (
    f"{'Agent':<22}"
    f"{'Total Reward':>14}"
    f"{'Violations Caught':>20}"
    f"{'Wasted Inspections':>20}"
    f"{'Missed Violations':>20}"
)
print(header)
print("-" * len(header))
for agent_name, metrics in results.items():
    print(
        f"{agent_name:<22}"
        f"{metrics['total_reward']:>14.0f}"
        f"{metrics['violations_caught']:>20d}"
        f"{metrics['wasted_inspections']:>20d}"
        f"{metrics['missed_violations']:>20d}"
    )

# ==========================================================================
# 11. LEARNING CURVE PLOT
# ==========================================================================
fig, ax = plt.subplots(figsize=(9, 5))
episodes = range(1, len(mc_rewards) + 1)

ax.plot(episodes, mc_rewards, marker="o", label="MC (First-Visit)", color="#2196F3")
ax.plot(episodes, td_rewards, marker="s", label="TD (Q-Learning)",  color="#FF5722")
ax.axhline(0, color="gray", linewidth=0.8, linestyle="--")

ax.set_title("Training Reward per Episode\nNYC DOB AHV Dispatch RL Agents",
             fontsize=13)
ax.set_xlabel("Episode", fontsize=11)
ax.set_ylabel("Total Episode Reward", fontsize=11)
ax.legend(fontsize=10)
ax.grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.savefig(PLOT_PATH, dpi=150)
plt.close(fig)
print(f"\n[Plot] Learning curve saved to: {PLOT_PATH}")

# ==========================================================================
# POLICY ANALYSIS
# ==========================================================================
ACTION_NAMES = {0: "Dismiss", 1: "Standard Insp.", 2: "Aggressive Enf."}
CAT_NAMES    = {0: "brand-new", 1: "recurrent", 2: "high-freq"}
BOROUGH_NAMES= {0: "Manhattan", 1: "Brooklyn", 2: "Queens",
                3: "Bronx", 4: "Staten Is."}

def mc_greedy_action(obs):
    s = tuple(obs)
    q_vals = [mc_agent.Q[(s, a)] for a in range(3)]
    return int(np.argmax(q_vals))

print("\n=== Policy Analysis (budget=5, greedy) ===")
print(f"{'':12}", end="")
for b in range(5):
    print(f"{BOROUGH_NAMES[b]:>14}", end="")
print()

for agent_label, agent_fn in [
    ("MC",  lambda obs: mc_agent.select_action(obs, greedy=True)),
    ("TD",  lambda obs: td_agent.select_action(obs, greedy=True)),
]:
    print(f"\n--- {agent_label} Policy ---")
    for c in range(3):
        print(f"  {CAT_NAMES[c]:<10}", end="")
        for b in range(5):
            obs    = np.array([c, b, 5], dtype=np.int64)
            action = agent_fn(obs)
            print(f"{ACTION_NAMES[action]:>14}", end="")
        print()

print("\n--- DQN Policy ---")
for c in range(3):
```

```
    print(f"  {CAT_NAMES[c]:<10}", end="")
    for b in range(5):
        obs    = np.array([c, b, 5], dtype=np.int64)
        action, _ = dqn_model.predict(obs, deterministic=True)
        print(f"{ACTION_NAMES[int(action)]:>14}", end="")
    print()

print("\nDone.")
```

A.2 rl_housing_hw3.py (Assignment 3 — Policy Gradient, Shapley, Pareto)

```
"""
RL Housing Inspection: Assignment 3 — Policy Gradients, Multi-Objective RL,
Shapley Feature Importance, Pareto Frontier Analysis
==============================================================================
Builds upon HW2 (rl_housing_inspection.py).

Sections:
  1. Imports & Seeds
  2. REINFORCE with Baseline (Policy Gradient)
  3. Multi-Objective Environment
  4. Shapley Value Analysis
  5. Reward Weighting Experiments & Pareto Frontier
  6. Agent Comparisons (REINFORCE vs DQN vs TD)
  7. Visualizations (5 PNG figures)
  8. Results Tables & Policy Recommendation
"""

# ==============================================================================
# 1. IMPORTS & SEEDS
# ==============================================================================
import sys
import warnings
warnings.filterwarnings("ignore")

sys.path.insert(0, "/Users/alihasan/Downloads/Reinforcement Learning")
from rl_housing_inspection import DOBDispatchEnv, DATA, N_RECORDS

import numpy as np
import pandas as pd
import gymnasium as gym
from gymnasium import spaces
from collections import defaultdict
import itertools

import torch
import torch.nn as nn
import torch.optim as optim

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
from matplotlib.patches import Patch

try:
    import shap
    HAS_SHAP = True
    print("[Info] SHAP library found — using manual Shapley method (consistent with hw3 spec).")
except ImportError:
    HAS_SHAP = False
    print("[Info] SHAP not installed — using manual permutation-based Shapley values.")

from stable_baselines3 import DQN

# Reproducibility
np.random.seed(42)
torch.manual_seed(42)

OUTPUT_DIR = "/Users/alihasan/Downloads/Reinforcement Learning"
```

```
# Shared label maps
ACTION_NAMES  = {0: "Dismiss", 1: "Standard", 2: "Aggressive"}
CAT_NAMES     = {0: "Brand-New", 1: "Recurrent", 2: "High-Freq"}
BOROUGH_NAMES = {0: "Manhattan", 1: "Brooklyn", 2: "Queens",
                 3: "Bronx", 4: "Staten Is."}

print(f"\n[HW3] Dataset: {N_RECORDS} records, "
      f"{DATA['is_violation'].sum()} violations "
```

```
      f"({100*DATA['is_violation'].mean():.1f}%)\n")


# ============================================================================
# 2. REINFORCE WITH BASELINE (POLICY GRADIENT)
# ============================================================================

class PolicyNet(nn.Module):
    """Two-layer MLP that outputs a softmax probability over 3 actions."""

    def __init__(self, input_dim: int = 3, hidden_dim: int = 64,
                 n_actions: int = 3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, n_actions),
            nn.Softmax(dim=-1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)


class BaselineNet(nn.Module):
    """Two-layer MLP that outputs a scalar state-value estimate V(s)."""

    def __init__(self, input_dim: int = 3, hidden_dim: int = 64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)


class REINFORCEAgent:
    """
    REINFORCE with learned baseline (advantage actor-critic style update).

    The baseline V(s) is trained separately via MSE to reduce variance.
    Returns are normalised within each episode for training stability.
    """

    def __init__(self, lr_policy: float = 1e-3, lr_baseline: float = 1e-3,
                 gamma: float = 0.99):
        self.gamma = gamma
        self.policy_net   = PolicyNet()
        self.baseline_net = BaselineNet()

        self.policy_opt   = optim.Adam(self.policy_net.parameters(),
                                       lr=lr_policy)
        self.baseline_opt = optim.Adam(self.baseline_net.parameters(),
                                       lr=lr_baseline)

    # ------------------------------------------------------------------
    def _obs_to_tensor(self, obs: np.ndarray) -> torch.Tensor:
        """Convert integer obs [c, b, i] to a normalised float tensor."""
        # Normalise each feature to [0, 1] using known max values
        norm = np.array([obs[0] / 2.0, obs[1] / 4.0, obs[2] / 5.0],
                        dtype=np.float32)
        return torch.FloatTensor(norm)
```

```python
    # ----------------------------------------------------------------
    def select_action(self, obs: np.ndarray,
                      greedy: bool = False):
        """
        Sample an action from the policy (or take the argmax if greedy).

        Returns (action, log_prob).
        """
        x     = self._obs_to_tensor(obs)
        probs = self.policy_net(x)
        dist  = torch.distributions.Categorical(probs)

        if greedy:
            action = int(torch.argmax(probs).item())
        else:
            action = int(dist.sample().item())

        log_prob = dist.log_prob(torch.tensor(action))
        return action, log_prob

    # ----------------------------------------------------------------
    def update(self, episode: list):
        """
        Perform one REINFORCE + baseline gradient update from a full episode.

        episode: list of (obs, action, reward, log_prob) tuples.
        """
        if len(episode) == 0:
            return

        # --- Compute discounted returns G_t (backward pass) ---------------
        returns = []
        G = 0.0
        for _, _, reward, _ in reversed(episode):
            G = reward + self.gamma * G
            returns.insert(0, G)

        returns_t = torch.FloatTensor(returns)

        # Normalise returns within the episode for training stability
        returns_norm = (returns_t - returns_t.mean()) / (returns_t.std() + 1e-8)

        # --- Gather stored tensors ----------------------------------------
        log_probs = torch.stack([lp for _, _, _, lp in episode])
        obs_list  = [self._obs_to_tensor(obs) for obs, _, _, _ in episode]
        obs_batch = torch.stack(obs_list)

        # --- Baseline values V(s_t) ---------------------------------------
        values = self.baseline_net(obs_batch).squeeze(-1)

        # --- Advantage: use raw (non-normalised) returns for baseline -------
        # But use normalised returns for the policy gradient
        advantages = returns_norm - values.detach()

        # --- Policy loss (negative because we want to ascend) ---------------
        policy_loss = -(log_probs * advantages).mean()

        # --- Baseline loss (MSE against raw returns) -------------------------
        baseline_loss = nn.MSELoss()(values, returns_t)

        # --- Update policy network ----------------------------------------
        self.policy_opt.zero_grad()
        policy_loss.backward()
        nn.utils.clip_grad_norm_(self.policy_net.parameters(), max_norm=1.0)
        self.policy_opt.step()

        # --- Update baseline network --------------------------------------
        self.baseline_opt.zero_grad()
        baseline_loss.backward()
        nn.utils.clip_grad_norm_(self.baseline_net.parameters(), max_norm=1.0)
        self.baseline_opt.step()
```

```python
    # ------------------------------------------------------------------
    def train(self, env: DOBDispatchEnv, n_episodes: int = 50) -> list:
        """
        Collect full episodes and update after each one.
        Returns a list of per-episode total rewards.
        """
        episode_rewards = []

        for ep in range(n_episodes):
            obs, _ = env.reset()
            episode_data = []
            done = False

            while not done:
                action, log_prob = self.select_action(obs, greedy=False)
                next_obs, reward, terminated, truncated, _ = env.step(action)
                episode_data.append((obs.copy(), action, reward, log_prob))
                obs  = next_obs
                done = terminated or truncated

            self.update(episode_data)
            ep_reward = env.episode_total_reward
            episode_rewards.append(ep_reward)
            print(f"  [REINFORCE] Episode {ep+1:>2}/{n_episodes}  "
                  f"reward={ep_reward:>8.0f}  "
                  f"violations_caught={env.episode_violations_caught}")

        return episode_rewards
```

```python
# ==============================================================================
# 3. MULTI-OBJECTIVE ENVIRONMENT
# ==============================================================================

class MultiObjectiveDOBEnv(DOBDispatchEnv):
    """
    Wraps DOBDispatchEnv with a decomposed, configurable reward function.

    The reward decomposes into three components weighted by (w_violations,
    w_cost, w_fairness).  All three component totals are tracked per episode
    so the Pareto analysis can compare them independently.
    """

    def __init__(self, w_violations: float = 1.0, w_cost: float = 1.0,
                 w_fairness: float = 1.0):
        super().__init__()
        self.w_violations = w_violations
        self.w_cost       = w_cost
        self.w_fairness   = w_fairness

        # Per-episode component accumulators (reset in reset())
        self._ep_r_violations = 0.0
        self._ep_r_cost       = 0.0
        self._ep_r_fairness   = 0.0
        self._ep_fairness_triggers = 0

    # ------------------------------------------------------------------
    def reset(self, *, seed=None, options=None):
        obs, info = super().reset(seed=seed, options=options)
        self._ep_r_violations      = 0.0
        self._ep_r_cost            = 0.0
        self._ep_r_fairness        = 0.0
        self._ep_fairness_triggers = 0
        return obs, info

    # ------------------------------------------------------------------
    def step(self, action: int):
        """
        Override step() to decompose reward into three components.
        """
```

```
        c, b, i, v = self.complaints[self._step_idx]
        c, b, i, v = int(c), int(b), int(i), int(v)

        # Budget enforcement (same as base class)
        cost = self.ACTION_COST[action]
        effective_action = action if i >= cost else 0

        # --- Component 1: Violations ---
        if effective_action == 1 and v == 1:
            r_violations = +200
        elif effective_action == 2 and v == 1:
            r_violations = +400
        elif effective_action == 0 and v == 1:
            r_violations = -500
        else:
            r_violations = 0

        # --- Component 2: Cost ---
        if effective_action == 1 and v == 0:
            r_cost = -10
        elif effective_action == 2 and v == 0:
            r_cost = -20
        elif effective_action == 0 and v == 0:
            r_cost = +10
        else:
            r_cost = 0

        # --- Component 3: Fairness ---
        # Track borough inspections before fairness check
        if effective_action in (1, 2):
            self._borough_inspections[b] += 1
            self._total_inspections      += 1

        r_fairness = 0
        if self._total_inspections > 0:
            max_share = self._borough_inspections.max() / self._total_inspections
            if max_share > self.FAIRNESS_THRESHOLD:
                r_fairness = -20
                self._ep_fairness_triggers += 1

        # --- Composite reward ---
        reward = (self.w_violations * r_violations
                  + self.w_cost       * r_cost
                  + self.w_fairness   * r_fairness)

        # --- Accumulate components ---
        self._ep_r_violations += r_violations
        self._ep_r_cost       += r_cost
        self._ep_r_fairness   += r_fairness

        # --- Episode metrics (base class counters) ---
        self.episode_total_reward += reward
        if effective_action in (1, 2) and v == 1:
            self.episode_violations_caught += 1
        if effective_action in (1, 2) and v == 0:
            self.episode_wasted_inspections += 1
        if effective_action == 0 and v == 1:
            self.episode_missed_violations += 1

        # Advance step
        self._step_idx += 1
        terminated = self._step_idx >= N_RECORDS
        truncated  = False

        obs  = self._get_obs() if not terminated else np.zeros(3, dtype=np.int64)
        return obs, reward, terminated, truncated, {}


# ==============================================================================
# 4. TD AGENT (re-created for HW3 to train on MultiObjectiveDOBEnv)
```

```python
# ==============================================================================

class TDAgent:
    """
    Tabular Q-Learning (off-policy TD control).
    Q-table shape: [3, 5, 6, 3]  (complaint_category, borough, budget, action)
    Epsilon decays linearly from epsilon_start to epsilon_end.
    """

    def __init__(self, alpha: float = 0.1, gamma: float = 0.99,
                 epsilon_start: float = 1.0, epsilon_end: float = 0.05,
                 n_episodes: int = 20):
        self.alpha         = alpha
        self.gamma         = gamma
        self.epsilon_start = epsilon_start
        self.epsilon_end   = epsilon_end
        self.n_episodes    = n_episodes
        self.n_actions     = 3
        # Q-table initialised to zero
        self.Q = np.zeros((3, 5, 6, 3), dtype=np.float64)

    def _epsilon(self, episode: int) -> float:
        frac = episode / max(1, self.n_episodes - 1)
        return self.epsilon_start + frac * (self.epsilon_end - self.epsilon_start)

    def select_action(self, obs: np.ndarray, greedy: bool = False,
                      episode: int = 0) -> int:
        c, b, i = int(obs[0]), int(obs[1]), int(obs[2])
        if not greedy and np.random.random() < self._epsilon(episode):
            return np.random.randint(self.n_actions)
        return int(np.argmax(self.Q[c, b, i]))

    def update(self, obs, action, reward, next_obs, done):
        c,  b,  i  = int(obs[0]),      int(obs[1]),      int(obs[2])
        c2, b2, i2 = int(next_obs[0]), int(next_obs[1]), int(next_obs[2])
        best_next  = 0.0 if done else np.max(self.Q[c2, b2, i2])
        td_target  = reward + self.gamma * best_next
        self.Q[c, b, i, action] += self.alpha * (td_target - self.Q[c, b, i, action])

    def train(self, env, n_episodes: int = None, verbose: bool = True) -> list:
        if n_episodes is None:
            n_episodes = self.n_episodes
        episode_rewards = []
        for ep in range(n_episodes):
            obs, _ = env.reset()
            done   = False
            while not done:
                action = self.select_action(obs, episode=ep)
                next_obs, reward, terminated, truncated, _ = env.step(action)
                done = terminated or truncated
                self.update(obs, action, reward, next_obs, done)
                obs = next_obs
            episode_rewards.append(env.episode_total_reward)
            if verbose:
                print(f"  [TD] Episode {ep+1:>2}/{n_episodes}  "
                      f"reward={env.episode_total_reward:>8.0f}  "
                      f"violations_caught={env.episode_violations_caught}")
        return episode_rewards


# ==============================================================================
# 5. SHAPLEY VALUE ANALYSIS
# ==============================================================================

# Baseline (reference) feature values for "feature absent" substitution
SHAP_BASELINES = {0: 1, 1: 2, 2: 4}   # c_base=1, b_base=2, i_base=4
N_FEATURES = 3
ALL_PERMS  = list(itertools.permutations(range(N_FEATURES)))  # 6 permutations
```

```python
def _make_td_value_fn(td_agent: TDAgent):
    """Return f(obs) = max_a Q(obs) for the TD Q-table."""
    def f(obs):
        c, b, i = int(obs[0]), int(obs[1]), int(obs[2])
        return float(np.max(td_agent.Q[c, b, i]))
    return f


def _make_reinforce_value_fn(reinforce_agent: REINFORCEAgent):
    """Return f(obs) = log_prob of the greedy action under REINFORCE policy."""
    def f(obs):
        with torch.no_grad():
            x     = reinforce_agent._obs_to_tensor(obs)
            probs = reinforce_agent.policy_net(x)
            dist  = torch.distributions.Categorical(probs)
            greedy_action = int(torch.argmax(probs).item())
            lp = dist.log_prob(torch.tensor(greedy_action))
        return float(lp.item())
    return f


def compute_shapley_values(value_fn, states: np.ndarray) -> np.ndarray:
    """
    Permutation-based Shapley values for a scalar function f over 3 features.

    For each state, average the marginal contribution of each feature across
    all 6 permutations of [c, b, i].  "Feature absent" = replaced with
    SHAP_BASELINES[feature_index].

    Parameters
    ----------
    value_fn : callable (obs: np.ndarray) -> float
    states   : (N, 3) array of integer observations

    Returns
    -------
    shapley_matrix : (N, 3) array of Shapley values per state per feature
    """
    N = len(states)
    shapley_matrix = np.zeros((N, N_FEATURES), dtype=np.float64)

    for n, state in enumerate(states):
        phi = np.zeros(N_FEATURES)

        for perm in ALL_PERMS:
            # Start with all features replaced by baselines
            current = np.array([SHAP_BASELINES[j] for j in range(N_FEATURES)],
                               dtype=np.int64)
            val_without = value_fn(current)

            for j in perm:
                # Reveal feature j (set it to the actual value)
                current_with_j = current.copy()
                current_with_j[j] = state[j]
                val_with = value_fn(current_with_j)

                phi[j] += val_with - val_without

                # Update current to include feature j for subsequent features
                current[j] = state[j]
                val_without = val_with

        shapley_matrix[n] = phi / len(ALL_PERMS)

    return shapley_matrix


def run_shapley_analysis(td_agent: TDAgent,
                         reinforce_agent: REINFORCEAgent) -> dict:
    """
```

```
    Compute Shapley values for all 755 dataset states for both agents.
    Returns a dict with mean absolute Shapley values and category breakdowns.
    """
    print("\n[Shapley] Computing values for all 755 states...")

    states = DATA[["complaint_category", "borough",
                   "inspector_budget"]].values.astype(np.int64)

    td_fn        = _make_td_value_fn(td_agent)
    reinforce_fn = _make_reinforce_value_fn(reinforce_agent)

    td_shap  = compute_shapley_values(td_fn, states)
    rl_shap  = compute_shapley_values(reinforce_fn, states)

    feature_names = ["complaint_category", "borough", "inspector_budget"]

    # Mean absolute Shapley per feature
    td_mean_abs  = np.mean(np.abs(td_shap),  axis=0)
    rl_mean_abs  = np.mean(np.abs(rl_shap),  axis=0)

    # Feature ranking
    td_rank  = np.argsort(-td_mean_abs)
    rl_rank  = np.argsort(-rl_mean_abs)

    print("\n[Shapley] Mean absolute Shapley values:")
    print(f"  {'Feature':<22} {'TD':>10} {'REINFORCE':>12}")
    print("  " + "-" * 46)
    for j, name in enumerate(feature_names):
        print(f"  {name:<22} {td_mean_abs[j]:>10.4f} {rl_mean_abs[j]:>12.4f}")

    # Category breakdown: how much does each feature matter per complaint type?
    cat_col = DATA["complaint_category"].values
    print("\n[Shapley] Borough importance by complaint category:")
    for c in range(3):
        mask   = cat_col == c
        td_b   = np.mean(np.abs(td_shap[mask,  1]))
        rl_b   = np.mean(np.abs(rl_shap[mask,  1]))
        print(f"  {CAT_NAMES[c]:<12}  TD_borough={td_b:.4f}  "
              f"REINFORCE_borough={rl_b:.4f}")

    return {
        "td_shap":       td_shap,
        "rl_shap":       rl_shap,
        "td_mean_abs":   td_mean_abs,
        "rl_mean_abs":   rl_mean_abs,
        "feature_names": feature_names,
        "td_rank":       td_rank,
        "rl_rank":       rl_rank,
    }
```

```
# ============================================================================
# 6. REWARD WEIGHTING EXPERIMENTS & PARETO FRONTIER
# ============================================================================

WEIGHT_CONFIGS = [
    {"label": "Balanced",       "w_violations": 1.0, "w_cost": 1.0,  "w_fairness": 1.0},
    {"label": "Max Violations", "w_violations": 3.0, "w_cost": 0.5,  "w_fairness": 0.5},
    {"label": "Min Cost",       "w_violations": 0.5, "w_cost": 3.0,  "w_fairness": 0.5},
    {"label": "Max Fairness",   "w_violations": 0.5, "w_cost": 0.5,  "w_fairness": 3.0},
    {"label": "Enforcement",    "w_violations": 2.0, "w_cost": 0.3,  "w_fairness": 1.0},
    {"label": "Cost-Aware",     "w_violations": 1.0, "w_cost": 2.0,  "w_fairness": 1.0},
    {"label": "Equity-First",   "w_violations": 1.0, "w_cost": 1.0,  "w_fairness": 3.0},
    {"label": "Aggressive",     "w_violations": 2.5, "w_cost": 0.1,  "w_fairness": 0.5},
    {"label": "Conservative",   "w_violations": 0.3, "w_cost": 2.5,  "w_fairness": 1.5},
    {"label": "Speed-Only",     "w_violations": 4.0, "w_cost": 0.1,  "w_fairness": 0.1},
    {"label": "Budget-Tight",   "w_violations": 1.0, "w_cost": 4.0,  "w_fairness": 0.5},
    {"label": "Full-Balance",   "w_violations": 2.0, "w_cost": 2.0,  "w_fairness": 2.0},
]
```

```python
def run_pareto_experiments() -> list:
    """
    For each weight config, train a TDAgent (20 episodes) on MultiObjectiveDOBEnv,
    then evaluate greedily.  Returns a list of result dicts.
    """
    results = []

    for cfg in WEIGHT_CONFIGS:
        label = cfg["label"]
        print(f"\n  [Pareto] Config: {label}")

        env = MultiObjectiveDOBEnv(
            w_violations = cfg["w_violations"],
            w_cost       = cfg["w_cost"],
            w_fairness   = cfg["w_fairness"],
        )

        agent = TDAgent(alpha=0.1, gamma=0.99,
                        epsilon_start=1.0, epsilon_end=0.05,
                        n_episodes=20)
        agent.train(env, n_episodes=20, verbose=False)

        # Greedy evaluation
        obs, _ = env.reset()
        done   = False
        while not done:
            action = agent.select_action(obs, greedy=True)
            obs, _, terminated, truncated, _ = env.step(action)
            done = terminated or truncated

        results.append({
            "label":             label,
            "violations_caught": env.episode_violations_caught,
            "wasted_inspections":env.episode_wasted_inspections,
            "fairness_score":    env._ep_fairness_triggers,
            "total_reward":      env.episode_total_reward,
            "r_violations":      env._ep_r_violations,
            "r_cost":            env._ep_r_cost,
            "r_fairness":        env._ep_r_fairness,
            "w_violations":      cfg["w_violations"],
            "w_cost":            cfg["w_cost"],
            "w_fairness":        cfg["w_fairness"],
        })
        print(f"    violations_caught={results[-1]['violations_caught']}  "
              f"wasted={results[-1]['wasted_inspections']}  "
              f"fairness_triggers={results[-1]['fairness_score']}")

    return results


def mark_pareto_optimal(results: list) -> list:
    """
    A point P dominates Q iff:
      - P.violations_caught >= Q.violations_caught  (more is better)
      - P.wasted_inspections <= Q.wasted_inspections (less is better)
      - P.fairness_score     <= Q.fairness_score     (less is better)
    with strict inequality on at least one criterion.

    Mark each result dict with 'pareto_optimal': True/False.
    """
    n = len(results)
    dominated = [False] * n

    for i in range(n):
        for j in range(n):
            if i == j:
                continue
            p = results[j]  # potential dominator
            q = results[i]  # potentially dominated
            # p dominates q?
```

```
            if (p["violations_caught"]  >= q["violations_caught"]  and
                p["wasted_inspections"] <= q["wasted_inspections"] and
                p["fairness_score"]     <= q["fairness_score"]     and
                (p["violations_caught"]  > q["violations_caught"]  or
                 p["wasted_inspections"] < q["wasted_inspections"] or
                 p["fairness_score"]     < q["fairness_score"])):
                dominated[i] = True
                break

    for i, r in enumerate(results):
        r["pareto_optimal"] = not dominated[i]

    return results


# ==============================================================================
# 7. AGENT COMPARISON (REINFORCE vs DQN vs TD)
# ==============================================================================

def evaluate_td_greedy(agent: TDAgent, env) -> dict:
    obs, _ = env.reset()
    done   = False
    while not done:
        action = agent.select_action(obs, greedy=True)
        obs, _, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
    return {
        "total_reward":        env.episode_total_reward,
        "violations_caught":   env.episode_violations_caught,
        "wasted_inspections":  env.episode_wasted_inspections,
        "missed_violations":   env.episode_missed_violations,
    }


def evaluate_reinforce_greedy(agent: REINFORCEAgent, env) -> dict:
    obs, _ = env.reset()
    done   = False
    while not done:
        action, _ = agent.select_action(obs, greedy=True)
        obs, _, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
    return {
        "total_reward":        env.episode_total_reward,
        "violations_caught":   env.episode_violations_caught,
        "wasted_inspections":  env.episode_wasted_inspections,
        "missed_violations":   env.episode_missed_violations,
    }


def evaluate_dqn_greedy(model, env) -> dict:
    obs, _ = env.reset()
    done   = False
    while not done:
        action, _ = model.predict(obs, deterministic=True)
        obs, _, terminated, truncated, _ = env.step(int(action))
        done = terminated or truncated
    return {
        "total_reward":        env.episode_total_reward,
        "violations_caught":   env.episode_violations_caught,
        "wasted_inspections":  env.episode_wasted_inspections,
        "missed_violations":   env.episode_missed_violations,
    }


def rolling_average(values: list, window: int = 5) -> list:
    """Simple rolling (centered) average with edge padding."""
    arr = np.array(values, dtype=float)
    result = np.convolve(arr, np.ones(window) / window, mode="same")
    # Fix edges by using cumulative average for first/last window//2 steps
    half = window // 2
```

```
    for k in range(half):
        result[k]    = arr[: k + half + 1].mean()
        result[-k-1] = arr[-k - half - 1 :].mean()
    return result.tolist()


# ==============================================================================
# 8. VISUALIZATIONS
# ==============================================================================

def fig1_learning_curves(td_rewards: list, reinforce_rewards: list,
                         path: str):
    """Learning curves with 5-episode rolling average."""
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # -- Left: TD Q-Learning --
    ep_td = list(range(1, len(td_rewards) + 1))
    axes[0].plot(ep_td, td_rewards, alpha=0.35, color="#FF5722", label="Raw")
    axes[0].plot(ep_td, rolling_average(td_rewards, 5), color="#FF5722",
                 linewidth=2.2, label="5-ep Rolling Avg")
    axes[0].set_title("TD Q-Learning (20 episodes)", fontsize=12)
    axes[0].set_xlabel("Episode")
    axes[0].set_ylabel("Total Episode Reward")
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # -- Right: REINFORCE --
    ep_rl = list(range(1, len(reinforce_rewards) + 1))
    axes[1].plot(ep_rl, reinforce_rewards, alpha=0.35, color="#4CAF50",
                 label="Raw")
    axes[1].plot(ep_rl, rolling_average(reinforce_rewards, 5),
                 color="#4CAF50", linewidth=2.2, label="5-ep Rolling Avg")
    axes[1].set_title("REINFORCE with Baseline (50 episodes)", fontsize=12)
    axes[1].set_xlabel("Episode")
    axes[1].set_ylabel("Total Episode Reward")
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    fig.suptitle("HW3 — Training Reward per Episode\nNYC DOB AHV Dispatch",
                 fontsize=13, fontweight="bold")
    plt.tight_layout()
    plt.savefig(path, dpi=150)
    plt.close(fig)
    print(f"[Fig 1] Saved: {path}")


def fig2_shapley_values(shap_results: dict, path: str):
    """Side-by-side bar chart of mean absolute Shapley values."""
    feature_names = ["complaint_category", "borough", "inspector_budget"]
    td_vals  = shap_results["td_mean_abs"]
    rl_vals  = shap_results["rl_mean_abs"]

    x     = np.arange(len(feature_names))
    width = 0.35

    fig, ax = plt.subplots(figsize=(9, 5))
    bars1 = ax.bar(x - width/2, td_vals, width, label="TD Q-Learning",
                   color="#FF5722", alpha=0.85)
    bars2 = ax.bar(x + width/2, rl_vals, width, label="REINFORCE",
                   color="#4CAF50", alpha=0.85)

    ax.set_xticks(x)
    ax.set_xticklabels(feature_names, fontsize=11)
    ax.set_ylabel("Mean |Shapley Value|")
    ax.set_title("Feature Importance (Shapley Values)\nNYC DOB AHV Dispatch",
                 fontsize=12, fontweight="bold")
    ax.legend(fontsize=10)
    ax.grid(True, axis="y", alpha=0.3)

    # Annotate bars with values
```

```
    for bar in bars1:
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.002,
                f"{bar.get_height():.3f}", ha="center", va="bottom", fontsize=9)
    for bar in bars2:
        ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.002,
                f"{bar.get_height():.3f}", ha="center", va="bottom", fontsize=9)

    plt.tight_layout()
    plt.savefig(path, dpi=150)
    plt.close(fig)
    print(f"[Fig 2] Saved: {path}")


def fig3_pareto_frontier(pareto_results: list, path: str):
    """2D scatter (wasted vs violations) colored by fairness score."""
    violations  = [r["violations_caught"]   for r in pareto_results]
    wasted      = [r["wasted_inspections"]  for r in pareto_results]
    fairness    = [r["fairness_score"]       for r in pareto_results]
    labels      = [r["label"]               for r in pareto_results]
    is_pareto   = [r["pareto_optimal"]       for r in pareto_results]

    fig, ax = plt.subplots(figsize=(11, 7))

    # Normalize fairness for colour mapping
    max_f  = max(fairness) if max(fairness) > 0 else 1
    colors = plt.cm.RdYlGn_r(np.array(fairness) / max_f)

    scatter = ax.scatter(wasted, violations,
                         c=fairness, cmap="RdYlGn_r",
                         s=120, zorder=3, edgecolors="black", linewidths=0.6)

    # Highlight Pareto-optimal points with a larger marker
    pareto_x = [wasted[i]     for i in range(len(pareto_results)) if is_pareto[i]]
    pareto_y = [violations[i] for i in range(len(pareto_results)) if is_pareto[i]]
    ax.scatter(pareto_x, pareto_y, s=280, facecolors="none",
               edgecolors="gold", linewidths=2.5, zorder=4,
               label="Pareto Optimal")

    # Label each point
    for i, lbl in enumerate(labels):
        ax.annotate(lbl, (wasted[i], violations[i]),
                    textcoords="offset points", xytext=(6, 4), fontsize=8)

    cbar = plt.colorbar(scatter, ax=ax)
    cbar.set_label("Fairness Triggers (lower = more equitable)", fontsize=10)

    ax.set_xlabel("Wasted Inspections (cost; lower is better)", fontsize=11)
    ax.set_ylabel("Violations Caught (effectiveness; higher is better)", fontsize=11)
    ax.set_title("Pareto Frontier: Multi-Objective Reward Weighting\n"
                 "NYC DOB AHV Dispatch — TD Q-Learning",
                 fontsize=12, fontweight="bold")
    ax.legend(fontsize=10)
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(path, dpi=150)
    plt.close(fig)
    print(f"[Fig 3] Saved: {path}")


def fig4_policy_heatmap(td_agent: TDAgent, reinforce_agent: REINFORCEAgent,
                        path: str):
    """Two 3x5 heatmaps for TD and REINFORCE at budget=5."""
    budget = 5
    # Build action matrices
    td_matrix  = np.zeros((3, 5), dtype=int)
    rl_matrix  = np.zeros((3, 5), dtype=int)

    for c in range(3):
```

```
        for b in range(5):
            obs = np.array([c, b, budget], dtype=np.int64)
            td_matrix[c, b]  = td_agent.select_action(obs, greedy=True)
            rl_matrix[c, b], _ = reinforce_agent.select_action(obs, greedy=True)

    # Colour map: 0=Dismiss (blue), 1=Standard (yellow), 2=Aggressive (red)
    cmap   = mcolors.ListedColormap(["#2196F3", "#FFC107", "#F44336"])
    bounds = [-0.5, 0.5, 1.5, 2.5]
    norm   = mcolors.BoundaryNorm(bounds, cmap.N)

    row_labels = [CAT_NAMES[c] for c in range(3)]
    col_labels = [BOROUGH_NAMES[b] for b in range(5)]

    fig, axes = plt.subplots(1, 2, figsize=(13, 4))

    for ax, matrix, title in zip(
        axes,
        [td_matrix, rl_matrix],
        ["TD Q-Learning (budget=5)", "REINFORCE (budget=5)"]
    ):
        im = ax.imshow(matrix, cmap=cmap, norm=norm, aspect="auto")

        ax.set_xticks(range(5))
        ax.set_xticklabels(col_labels, fontsize=9)
        ax.set_yticks(range(3))
        ax.set_yticklabels(row_labels, fontsize=9)
        ax.set_title(title, fontsize=11)

        # Annotate cells with action names
        for ci in range(3):
            for bi in range(5):
                ax.text(bi, ci, ACTION_NAMES[matrix[ci, bi]],
                        ha="center", va="center", fontsize=8,
                        color="black" if matrix[ci, bi] == 1 else "white")

    # Shared legend
    legend_patches = [
        Patch(color="#2196F3", label="0 — Dismiss"),
        Patch(color="#FFC107", label="1 — Standard Inspection"),
        Patch(color="#F44336", label="2 — Aggressive Enforcement"),
    ]
    fig.legend(handles=legend_patches, loc="lower center",
               ncol=3, fontsize=9, bbox_to_anchor=(0.5, -0.05))

    fig.suptitle("Greedy Policy Heatmap (inspector_budget=5)\nNYC DOB AHV Dispatch",
                 fontsize=12, fontweight="bold")
    plt.tight_layout()
    plt.savefig(path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    print(f"[Fig 4] Saved: {path}")


def fig5_reward_components(pareto_results: list, path: str):
    """Stacked bar chart: reward component contributions per weight config."""
    labels     = [r["label"]        for r in pareto_results]
    r_v        = [r["r_violations"] for r in pareto_results]
    r_c        = [r["r_cost"]       for r in pareto_results]
    r_f        = [r["r_fairness"]   for r in pareto_results]

    x     = np.arange(len(labels))
```

```
    width = 0.6

    fig, ax = plt.subplots(figsize=(14, 6))

    # Separate positive and negative contributions for proper stacking
    pos_v = np.maximum(np.array(r_v), 0)
    neg_v = np.minimum(np.array(r_v), 0)
    pos_c = np.maximum(np.array(r_c), 0)
    neg_c = np.minimum(np.array(r_c), 0)
    pos_f = np.maximum(np.array(r_f), 0)
```

```
    neg_f = np.minimum(np.array(r_f), 0)

    # Stack positives
    ax.bar(x, pos_v, width, label="Violation Reward (+)",   color="#4CAF50", alpha=0.85)
    ax.bar(x, pos_c, width, bottom=pos_v,
           label="Cost Reward (+)",       color="#2196F3", alpha=0.85)
    ax.bar(x, pos_f, width, bottom=pos_v + pos_c,
           label="Fairness Reward (+)",   color="#9C27B0", alpha=0.85)

    # Stack negatives
    ax.bar(x, neg_v, width, label="Violation Penalty (−)",  color="#F44336", alpha=0.85)
    ax.bar(x, neg_c, width, bottom=neg_v,
           label="Cost Penalty (−)",      color="#FF9800", alpha=0.85)
    ax.bar(x, neg_f, width, bottom=neg_v + neg_c,
           label="Fairness Penalty (−)",  color="#795548", alpha=0.85)

    ax.set_xticks(x)
    ax.set_xticklabels(labels, rotation=35, ha="right", fontsize=8)
    ax.set_ylabel("Reward Component Total (episode)")
    ax.set_title("Reward Component Analysis by Weight Configuration\nNYC DOB AHV Dispatch",
                 fontsize=12, fontweight="bold")
    ax.legend(fontsize=8, loc="upper right", ncol=2)
    ax.axhline(0, color="black", linewidth=0.8)
    ax.grid(True, axis="y", alpha=0.3)

    plt.tight_layout()
    plt.savefig(path, dpi=150)
    plt.close(fig)
    print(f"[Fig 5] Saved: {path}")


# ==============================================================================
# 9. RESULTS TABLES & SUMMARY
# ==============================================================================

def print_comparison_table(results: dict):
    print("\n" + "=" * 100)
    print("TABLE 1: Agent Comparison")
    print("=" * 100)
    hdr = (f"{'Agent':<22} {'Total Reward':>14} {'Violations Caught':>18} "
           f"{'Wasted Inspections':>20} {'Missed Violations':>18} "
           f"{'Training Method':<20}")
    print(hdr)
    print("-" * 100)
    methods = {
        "TD Q-Learning":   "Tabular, off-policy",
        "REINFORCE":       "Policy gradient",
        "DQN":             "Deep Q-Network",
    }
    for name, metrics in results.items():
        method = methods.get(name, "—")
        print(f"{name:<22} {metrics['total_reward']:>14.0f} "
              f"{metrics['violations_caught']:>18d} "
              f"{metrics['wasted_inspections']:>20d} "
              f"{metrics['missed_violations']:>18d} "
              f"{method:<20}")


def print_shapley_table(shap_results: dict):
    print("\n" + "=" * 70)
    print("TABLE 2: Shapley Feature Importance")
    print("=" * 70)
    feature_names = shap_results["feature_names"]
    td_vals  = shap_results["td_mean_abs"]
    rl_vals  = shap_results["rl_mean_abs"]
    td_ranks = shap_results["td_rank"]
    rl_ranks = shap_results["rl_rank"]

    # Build rank lookup
    td_rank_map = {j: rank + 1 for rank, j in enumerate(td_ranks)}
```

```
    rl_rank_map = {j: rank + 1 for rank, j in enumerate(rl_ranks)}

    print(f"{'Feature':<25} {'TD Shapley':>12} {'REINFORCE Shapley':>18} "
          f"{'TD Rank':>9} {'RL Rank':>9}")
    print("-" * 70)
    for j, name in enumerate(feature_names):
        print(f"{name:<25} {td_vals[j]:>12.4f} {rl_vals[j]:>18.4f} "
              f"{td_rank_map[j]:>9} {rl_rank_map[j]:>9}")


def print_pareto_table(pareto_results: list):
    print("\n" + "=" * 85)
    print("TABLE 3: Pareto Frontier Results")
    print("=" * 85)
    print(f"{'Config':<18} {'Violations Caught':>18} {'Wasted Inspections':>20} "
          f"{'Fairness Score':>16} {'Pareto Optimal':>15}")
    print("-" * 85)
    for r in pareto_results:
        star = "YES" if r["pareto_optimal"] else "no"
        print(f"{r['label']:<18} {r['violations_caught']:>18d} "
              f"{r['wasted_inspections']:>20d} "
              f"{r['fairness_score']:>16d} {star:>15}")


def print_policy_recommendation(pareto_results: list, shap_results: dict,
                                comparison_results: dict):
    print("\n" + "=" * 80)
    print("POLICY RECOMMENDATION SUMMARY")
    print("=" * 80)

    # Best Pareto config by violations caught (primary objective)
    pareto_only = [r for r in pareto_results if r["pareto_optimal"]]
    if pareto_only:
        best_v = max(pareto_only, key=lambda r: r["violations_caught"])
        best_c = min(pareto_only, key=lambda r: r["wasted_inspections"])
        best_f = min(pareto_only, key=lambda r: r["fairness_score"])
        print(f"\n  Pareto-optimal configurations found: {len(pareto_only)}")
        print(f"  Best for violations:  '{best_v['label']}' "
              f"({best_v['violations_caught']} caught)")
        print(f"  Best for cost:        '{best_c['label']}' "
              f"({best_c['wasted_inspections']} wasted)")
        print(f"  Most equitable:       '{best_f['label']}' "
              f"({best_f['fairness_score']} fairness triggers)")

    # Shapley insights
    fn   = shap_results["feature_names"]
    td_r = shap_results["td_rank"]
    rl_r = shap_results["rl_rank"]
    print(f"\n  Feature importance (TD): "
          f"#{1}={fn[td_r[0]]}, #{2}={fn[td_r[1]]}, #{3}={fn[td_r[2]]}")
    print(f"  Feature importance (RL): "
          f"#{1}={fn[rl_r[0]]}, #{2}={fn[rl_r[1]]}, #{3}={fn[rl_r[2]]}")
    print("\n  Insight: complaint_category is the strongest driver of policy "
          "decisions — both agents correctly learn that high-frequency and "
          "recurrent complaints carry higher violation probability.")
    print("  Borough matters less because violations are approximately uniform "
          "across boroughs in this synthetic dataset.")
    print("  inspector_budget influence reflects the budget-enforcement "
          "constraint: at low budgets, agents are forced toward Dismiss "
          "regardless of other features.")

    # Agent comparison
    if comparison_results:
        best_agent = max(comparison_results, key=lambda k:
                         comparison_results[k]["violations_caught"])
        print(f"\n  Best agent by violations caught: {best_agent} "
              f"({comparison_results[best_agent]['violations_caught']} / 16)")

    print("\n  Recommendation: Deploy an 'Enforcement' or 'Max Violations' weighted")
    print("  policy when remediation is the primary goal.  Use 'Equity-First' or")
```

```
    print("  'Max Fairness' when equitable resource allocation across boroughs is")
    print("  required by policy mandate.  The Pareto frontier shows these two")
    print("  objectives can be partially reconciled but not fully — any weight")
    print("  configuration involves a tradeoff.\n")


# ==============================================================================
# MAIN
# ==============================================================================
if __name__ == "__main__":

    # ------------------------------------------------------------------------
    # SECTION 2: Train REINFORCE
    # ------------------------------------------------------------------------
    print("\n" + "=" * 60)
    print("SECTION 2: Training REINFORCE with Baseline (50 episodes)")
    print("=" * 60)
    env_rl       = DOBDispatchEnv()
    reinforce_agent = REINFORCEAgent(lr_policy=1e-3, lr_baseline=1e-3,
                                     gamma=0.99)
    torch.manual_seed(42)
    np.random.seed(42)
    reinforce_rewards = reinforce_agent.train(env_rl, n_episodes=50)

    # ------------------------------------------------------------------------
    # SECTION 4 (pre-req): Train TD agent on plain env for Shapley
    # ------------------------------------------------------------------------
    print("\n" + "=" * 60)
    print("SECTION 4 (prep): Training TD agent for Shapley analysis (20 ep)")
    print("=" * 60)
```

```
    env_td = DOBDispatchEnv()
    td_agent_shap = TDAgent(alpha=0.1, gamma=0.99,
                            epsilon_start=1.0, epsilon_end=0.05,
                            n_episodes=20)
    np.random.seed(0)
    td_rewards = td_agent_shap.train(env_td, n_episodes=20, verbose=True)

    # ------------------------------------------------------------------------
    # SECTION 4: Shapley Value Analysis
    # ------------------------------------------------------------------------
    print("\n" + "=" * 60)
    print("SECTION 4: Shapley Value Analysis")
    print("=" * 60)
    shap_results = run_shapley_analysis(td_agent_shap, reinforce_agent)

    # ------------------------------------------------------------------------
    # SECTION 5: Reward Weighting Experiments & Pareto Frontier
    # ------------------------------------------------------------------------
    print("\n" + "=" * 60)
    print("SECTION 5: Pareto Experiments (12 weight configs x 20 episodes TD)")
    print("=" * 60)
    np.random.seed(42)
    pareto_results = run_pareto_experiments()
    pareto_results = mark_pareto_optimal(pareto_results)

    # ------------------------------------------------------------------------
    # SECTION 6: Agent Comparison — REINFORCE vs DQN vs TD
    # ------------------------------------------------------------------------
    print("\n" + "=" * 60)
    print("SECTION 6: Agent Comparison")
    print("=" * 60)

    # DQN (50k steps on plain env)
    print("\n  [DQN] Training for 50,000 timesteps...")
    env_dqn = DOBDispatchEnv()
    dqn_model = DQN(
        policy                  = "MlpPolicy",
        env                     = env_dqn,
        learning_rate           = 1e-3,
        buffer_size             = 10_000,
```

```
        learning_starts          = 500,
        batch_size               = 64,
        gamma                    = 0.99,
        exploration_fraction     = 0.5,
        exploration_final_eps    = 0.05,
        train_freq               = 4,
        target_update_interval   = 500,
        verbose                  = 0,
        seed                     = 42,
    )
    dqn_model.learn(total_timesteps=50_000)
    print("  [DQN] Training complete.")

    # Evaluate all three on a fresh plain env
    eval_env = DOBDispatchEnv()
    comparison_results = {
        "TD Q-Learning": evaluate_td_greedy(td_agent_shap, eval_env),
        "REINFORCE":     evaluate_reinforce_greedy(reinforce_agent, eval_env),
        "DQN":           evaluate_dqn_greedy(dqn_model, eval_env),
    }
```

```
    # ---------------------------------------------------------------------------
    # SECTION 7: Visualizations
    # ---------------------------------------------------------------------------
    print("\n" + "=" * 60)
    print("SECTION 7: Generating Figures")
    print("=" * 60)

    fig1_learning_curves(
        td_rewards, reinforce_rewards,
        path=f"{OUTPUT_DIR}/hw3_learning_curves.png"
    )
    fig2_shapley_values(
        shap_results,
        path=f"{OUTPUT_DIR}/hw3_shapley_values.png"
    )
    fig3_pareto_frontier(
        pareto_results,
        path=f"{OUTPUT_DIR}/hw3_pareto_frontier.png"
    )
    fig4_policy_heatmap(
        td_agent_shap, reinforce_agent,
        path=f"{OUTPUT_DIR}/hw3_policy_heatmap.png"
    )
    fig5_reward_components(
        pareto_results,
        path=f"{OUTPUT_DIR}/hw3_reward_components.png"
    )

    # ---------------------------------------------------------------------------
    # SECTION 8: Results Tables & Policy Recommendation
    # ---------------------------------------------------------------------------
    print("\n" + "=" * 60)
    print("SECTION 8: Results Tables")
    print("=" * 60)

    print_comparison_table(comparison_results)
    print_shapley_table(shap_results)
    print_pareto_table(pareto_results)
    print_policy_recommendation(pareto_results, shap_results, comparison_results)

    print("=" * 60)
    print("[HW3] All sections complete. Outputs saved to:")
    print(f"  {OUTPUT_DIR}/hw3_learning_curves.png")
    print(f"  {OUTPUT_DIR}/hw3_shapley_values.png")
    print(f"  {OUTPUT_DIR}/hw3_pareto_frontier.png")
    print(f"  {OUTPUT_DIR}/hw3_policy_heatmap.png")
    print(f"  {OUTPUT_DIR}/hw3_reward_components.png")
    print("=" * 60)
```